

Problem Set 2

Hoa T. Vu

Fitting a Linear Model with PyTorch

The example below fits $\hat{y} = \theta_0 x + \theta_1$ to seven data points using SGD with step decay scheduling. The learning rate is multiplied by γ every k steps: $\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/k \rfloor}$.

```
import torch

X = torch.tensor([1, 2, 3, 4, 5, 6, 7], dtype=torch.float)
y = torch.tensor([38, 47, 53, 58, 63, 60, 62], dtype=torch.float)

theta = torch.zeros(2, requires_grad=True)
opt    = torch.optim.SGD([theta], lr=1e-2)
sched  = torch.optim.lr_scheduler.StepLR(opt, step_size=10, gamma=0.9)

for _ in range(1000):
    y_hat = theta[0]*X + theta[1]
    loss   = ((y - y_hat)**2).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()
    sched.step()           # multiply lr by 0.9 every 10 steps

print(theta)              # learned coefficients
print(loss.item())        # final MSE
```

To use stochastic gradient descent, pick one random sample per step instead of using all data:

```
for _ in range(1000):
    idx    = torch.randperm(len(X))[:4]           # random batch size 4
    y_hat  = theta[0]*X[idx] + theta[1]
    loss   = ((y[idx] - y_hat)**2).mean()         # mean over the batch
    opt.zero_grad()
    loss.backward()
    opt.step()
    sched.step()
```

Key points:

- `opt.zero_grad()`: clear gradients before each `.backward()`.
- `loss.backward()`: computes $\partial \mathcal{L} / \partial \theta_i$ for all i .
- `opt.step()`: applies $\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}$.
- `sched.step()`: called after `opt.step()`; multiplies lr by gamma every `step_size` iterations.

Problems

Problem 1: Cubic Fitting (PyTorch). Fit the model $\hat{y} = at^3 + bt^2 + ct + d$ to the data below using **PyTorch** and gradient descent.

t	y	t	y
-3.00	-76.39	0.43	-3.77
-2.57	-55.41	0.86	-3.61
-2.14	-34.20	1.29	3.21
-1.71	-21.28	1.71	8.84
-1.29	-18.66	2.14	16.65
-0.86	-12.17	2.57	32.36
-0.43	-6.37	3.00	49.94
0.00	-5.63		

Use the starter code:

- (a) Run gradient descent for at least 1000 iterations with a suitable learning rate.
- (b) Report the final values of a, b, c, d and the final loss.
- (c) Plot the data points and the fitted curve.

Problem 2: Cubic Fitting from Scratch. Repeat the fitting task from Problem 1, but implement gradient descent **without any libraries** — only built-in Python (lists, arithmetic, `math`).

- (a) Implement the update rule using only Python lists and loops. Run for at least 1000 iterations.
- (b) Report the final values of a, b, c, d and the final loss.
- (c) Compare your result to the PyTorch solution from Problem 1. Do they converge to the same values?

Problem 3: Subgradients.

(a) **ReLU:** $f(x) = \max(0, x)$

- Write down $\partial f(x)$ for all $x \in \mathbb{R}$.
- What value is conventionally used at $x = 0$ in practice, and why?

(b) **SReLU:** $f(x) = (\text{clip}(x, 0, 1))^2$, $\text{clip}(x, a, b) = \max(a, \min(b, x))$

- Write down $f(x)$ as a piecewise function.
- Write down $\partial f(x)$ for all x . Identify every point of non-differentiability and give the full subdifferential there.

Problem 4: Quadratic Fitting. The file `data.csv` contains 50 samples generated from

$$y = 3x_1^2 - 2x_2^2 + 0.5x_1 + x_2 + 20 + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1.5)$$

with $x_1, x_2 \in [-3, 3]$. Fit the model $\hat{y} = ax_1^2 + bx_2^2 + cx_1 + dx_2 + e$ using PyTorch. Starter code is in `q4_starter_code.py`.

- (a) Run SGD for 10 000 iterations with `lr=1e-3`. Print the loss every 1000 iterations.
- (b) Report the final values of a, b, c, d, e . How close are they to the true values 3, -2, 0.5, 1, 20?
- (c) Add a StepLR scheduler with `step_size=10`, `gamma=0.9`. Does it help convergence?

Python Programming

Problem 5: Lists. Given the list below, write Python code for each task.

```
scores = [88, 72, 95, 61, 84, 90]
```

- (a) Access the third score.
- (b) Slice the last three scores.
- (c) Append 77, then sort in descending order.
- (d) Remove all scores below 80 using a list comprehension.

Problem 6: Sieve of Eratosthenes. Find all primes up to n by repeatedly crossing out multiples:

1. Start with `is_prime = [True] * (n + 1)`; set `is_prime[0] = is_prime[1] = False`.
2. For each p from 2 to $\lfloor \sqrt{n} \rfloor$: if `is_prime[p]` is still `True`, then p is prime: mark all its multiples starting from p^2 as composite: `is_prime[p*p]`, `is_prime[p*p + p]`, `is_prime[p*p + 2*p]`, ... We start at p^2 because all smaller multiples of p have already been crossed out by earlier primes.
3. Collect all indices where `is_prime[i]` is `True`.

Tasks:

- (a) Implement `sieve(n)` and verify it returns `[2, 3, 5, 7, 11, 13]` for $n = 13$.
- (b) Use it to find the **1000th prime**.
- (c) Count how many primes are below 10^6 .